Ch. 4 Dynamic Programming

I. DP

A. a collection of algs. used to compute optimal policy given a perfect model as a Markov Decis. Proc. (MDP)

B. Unrealistic for real RL, but important theoretically

1. Usually approx. in real apps.

Ch. 4 → 2. For these problems, mostly assume finite MDP (all actions, states, rewards are finite). Aka no continuous distributions

II. Policy Eval

A. Given any arbitrary policy, w/ DP, you may calculate the value of every state

1. $v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma v_\pi(s')]$

- $\sum_a \pi(a|s)$: chance you do a certain action
- $\sum_{s',r} p(s',r|s,a)$: chance you end up in a certain future state, if random
- $[r + \gamma v_\pi(s')]$: expected return (value, G_{t+1}) of the resulting state

2. To use for policy eval, set all states' value to a guess (0) then update every state with above equation

B. If you keep updating, you converge to v_π. This is called iterative policy evaluation where $v_k \to v_\pi$ as $k \to \infty$.

III. Policy Improvement

A. We evaluate policy to determine when we should switch policy

1. One way, choose action such that $a \neq \pi(s)$ then use the policy for all steps after.

2. If we consider improving all states and actions, we consider

$$\pi'(s) = \text{argmax}_a q_\pi(s,a)$$

get a that maxes the q_π value

B. If the case where we try to greedily get the action that best maximizes value, we define policy improvement

1. Greedy policy denoted π'

2. If applied to the value function, we get the Bellman Equ. for optimal policy

$$v_{\pi'}(s) = \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma v_{\pi'}(s')]$$

Scanned with CamScanner

IV. Policy Iteration

A. Thus it follows that if you keep seeking improving policies, it will have ≥ value over time and thus

$$\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} v_{\pi_1} \xrightarrow{I} \dots \xrightarrow{I} \pi_* \xrightarrow{E} v_*$$

where E is eval & I is improvement

1. In a finite MDP w/ finite policies, you will converge to π_* and v_*
2. This is called <u>policy iteration</u>

V. Value Iteration

A. Policy iteration can be very long b/c of policy eval.

B. Policy eval being truncated after 1 step is <u>value iteration</u>

1. Does not lose convergence guarantee
2. $v_{k+1}(s) = \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma v_k(s')] \quad \forall s \in S$

C. In practice, a small change variable is kept over runs to stop the process when change goes under a threshold

VI. Async. Dynamic Programming

A. Major issue, DP discussed requires TOTAL state sweep.

B. ADP allows state values to be updated in any order, allowing choice of update. (Still need to backup other values later).

1. Avoiding sweeps may not result in less compute, but we are not locked in one trying to improve
2. Unimportant or infrequent update states may be avoided more.

C. This importantly allows updates to happen <u>while an agent is in an MDP</u> to <u>guide decisions.</u> Focuses on important states

VII. Generalized Policy Iteration

A. <u>GPI</u> refers to the two processes, policy eval and improvement, being let interact, regardless of the detail of the processes

B. Most all RL is GPI, where policy is improved by the value func. and the value func. driving towards the policy

C. When both stabilize, optimal policy that is greedy to the optimal value has been achieved. Making val consistent with policy removes the greedy, and a change to be greedy makes val. inconsistent.

Scanned with CamScanner